

```

1 // =====
2 // HEADERS – Libraries we need
3 // =====
4 #include <iostream> // cout, cin – for input/output to console
5 #include <fstream> // ofstream, ifstream – for reading/writing files
6 #include <string> // string type and string operations
7 #include <iomanip> // setw() – for formatting columns in output
8 #include <sstream> // stringstream – for splitting lines when loading files
9
10 using namespace std; // so we don't write std::cout every time
11
12
13 // =====
14 // CLASS 1: Patient
15 // Think of this as a "form" – every patient fills this same form
16 // =====
17 class Patient {
18 public:
19     // Data members (the fields on the form)
20     string id; // Auto-generated: P001, P002, P003...
21     string name; // Full name e.g. "Jean Baptiste"
22     int age; // Age between 0 and 120
23     string diagnosis; // e.g. "Malaria", "Diabetes"
24     string hospital; // Name of assigned hospital e.g. "CHUK"
25     int priority; // 1=Emergency, 2=Urgent, 3=Normal
26
27     // -----
28     // DEFAULT CONSTRUCTOR
29     // Called when we do: Patient p;
30     // Sets everything to zero/empty so no garbage values
31     // -----
32     Patient() {
33         id = "";
34         name = "";
35         age = 0;
36         diagnosis = "";
37         hospital = "";
38         priority = 0;
39     }
40
41     // -----
42     // PARAMETERIZED CONSTRUCTOR
43     // Called when we do: Patient p("P001", "Alice", 28, ...);
44     // The word "this->" means "this object's own variable"
45     // -----
46     Patient(string pid, string pname, int page,
47             string pdiag, string phosp, int ppri) {
48         id = pid;
49         name = pname;
50         age = page;
51         diagnosis = pdiag;
52         hospital = phosp;
53         priority = ppri;
54     }
55
56     // -----
57     // MEMBER FUNCTION: getPriorityLabel()
58     // Converts int priority to a readable word
59     // 1 ? "Emergency", 2 ? "Urgent", 3 ? "Normal"
60     // -----
61     string getPriorityLabel() {
62         if (priority == 1) return "Emergency";
63         if (priority == 2) return "Urgent";
64         return "Normal";
65     }
66

```

```

67 // -----
68 // MEMBER FUNCTION: display()
69 // Prints one patient row in a formatted table
70 // setw(n) means "use n characters wide" for alignment
71 // left = align text to the left side of the column
72 // -----
73 void display() {
74     cout << left
75         << setw(8) << id
76         << setw(22) << name
77         << setw(6) << age
78         << setw(15) << hospital
79         << setw(12) << getPriorityLabel()
80         << diagnosis << "\n";
81 }
82 };
83
84
85 // =====
86 // CLASS 2: Hospital
87 // Just stores the hospital's number and name
88 // =====
89 class Hospital {
90 public:
91     int    index;    // 1 to 5
92     string name;     // e.g. "King Faisal"
93
94     // Default constructor
95     Hospital() {
96         index = 0;
97         name  = "";
98     }
99
100    // Parameterized constructor
101    Hospital(int i, string n) {
102        index = i;
103        name  = n;
104    }
105 };
106
107
108 // =====
109 // CLASS 3: PatientSystem
110 // The MAIN class – manages everything
111 // =====
112 class PatientSystem {
113 private:
114     // ---- Data Storage ----
115     static const int MAX = 100;    // Maximum patients the system can hold
116     Patient  patients[MAX];        // Array of Patient objects – our main data store
117     int      patientCount;         // How many patients are currently in the array
118
119     Hospital hospitals[5];          // Fixed array of 5 hospitals
120     int      idCounter;             // Tracks last ID number used (starts at 0)
121     bool     sortedByID;           // Tracks if array is currently sorted by ID
122                                     // (Binary search only works when this is true)
123
124     // =====
125     // PRIVATE HELPER: generateID()
126     // Auto-generates IDs: P001, P002, ... P099, P100
127     // Called internally every time a patient is added
128     // =====
129     string generateID() {
130         idCounter++; // Increment the counter first
131
132         string id = "P"; // Start with the letter P

```

```

133
134 // Pad with zeros to keep IDs always 4 characters long
135 if (idCounter < 10) id += "00" + to_string(idCounter); // P001
136 else if (idCounter < 100) id += "0" + to_string(idCounter); // P042
137 else id += to_string(idCounter); // P100
138
139 return id;
140 }
141
142 // =====
143 // PRIVATE HELPER: getHospitalName()
144 // Takes hospital index (1-5), returns hospital name
145 // =====
146 string getHospitalName(int idx) {
147     for (int i = 0; i < 5; i++) {
148         if (hospitals[i].index == idx)
149             return hospitals[i].name;
150     }
151     return "Unknown"; // Safety fallback (should never reach here after validation)
152 }
153
154 // =====
155 // PRIVATE HELPER: printHeader()
156 // Prints the column headers for the patient table
157 // =====
158 void printHeader() {
159     cout << "\n" << left
160         << setw(8) << "ID"
161         << setw(22) << "Name"
162         << setw(6) << "Age"
163         << setw(15) << "Hospital"
164         << setw(12) << "Priority"
165         << "Diagnosis\n";
166     cout << string(75, '-') << "\n";
167     // string(75, '-') creates "-----" (75 dashes) as a separator line
168 }
169
170 public:
171 // =====
172 // CONSTRUCTOR – Runs once when PatientSystem object is created
173 // Sets starting values and loads the 5 hospitals
174 // =====
175 PatientSystem() {
176     patientCount = 0; // 0 patients at the start
177     idCounter = 0; // ID counter starts at 0 (first patient = P001)
178     sortedByID = false; // Array not sorted yet
179
180     // Load the 5 Rwanda hospitals into the fixed array
181     hospitals[0] = Hospital(1, "CHUK");
182     hospitals[1] = Hospital(2, "King Faisal");
183     hospitals[2] = Hospital(3, "Butaro");
184     hospitals[3] = Hospital(4, "Kibagabaga");
185     hospitals[4] = Hospital(5, "Ruhengeri");
186 }
187
188 // =====
189 // MENU 1: addPatient()
190 // Collects patient data from user, validates it, stores it
191 // =====
192 void addPatient() {
193     // Guard: check if array is full
194     if (patientCount >= MAX) {
195         cout << "System full! Cannot add more patients.\n";
196         return;
197     }
198 }

```

```

199
200     string name, diagnosis;
201     int age, hospIdx, priority;
202
203     cout << "\n===== ADD NEW PATIENT =====\n";
204
205     // -- Get Name --
206     cout << "Enter patient name: ";
207     cin.ignore(); // Clears leftover newline from previous cin >>
208     getline(cin, name); // getline allows spaces in the name
209
210     // -- Get Age WITH VALIDATION --
211     cout << "Enter age: ";
212     cin >> age;
213     while (age < 0 || age > 120) {
214         // Loop keeps asking until valid
215         cout << "Invalid age! Must be between 0 and 120. Try again: ";
216         cin >> age;
217     }
218
219     // -- Get Diagnosis --
220     cout << "Enter diagnosis: ";
221     cin.ignore();
222     getline(cin, diagnosis);
223
224     // -- Show hospitals, Get Hospital Index WITH VALIDATION --
225     cout << "\nAvailable Hospitals:\n";
226     for (int i = 0; i < 5; i++)
227         cout << " " << hospitals[i].index << ". " << hospitals[i].name << "\n";
228     cout << "Enter hospital index (1-5): ";
229     cin >> hospIdx;
230     while (hospIdx < 1 || hospIdx > 5) {
231         cout << "Invalid! Please enter a number from 1 to 5: ";
232         cin >> hospIdx;
233     }
234
235     // -- Get Priority WITH VALIDATION --
236     cout << "Enter priority (1=Emergency, 2=Urgent, 3=Normal): ";
237     cin >> priority;
238     while (priority < 1 || priority > 3) {
239         cout << "Invalid! Please enter 1, 2, or 3: ";
240         cin >> priority;
241     }
242
243     // -- Create Patient and Store --
244     string newID = generateID(); // Auto-generates e.g. P004
245     string hospName = getHospitalName(hospIdx); // Gets name from index
246
247     // Store the new Patient object at position patientCount in the array
248     patients[patientCount] = Patient(newID, name, age, diagnosis, hospName, priority);
249     patientCount++; // Increase the count
250     sortedByID = false; // Array is no longer sorted after adding
251
252     cout << "\nPatient added successfully. ID: " << newID << "\n";
253 }
254
255
256 // =====
257 // MENU 2: displayAll()
258 // Shows every patient in a formatted table
259 // =====
260 void displayAll() {
261     if (patientCount == 0) {
262         cout << "\nNo patients in the system.\n";
263         return;
264     }

```

```

265
266     cout << "\n===== ALL PATIENTS (" << patientCount << " total) =====";
267     printHeader(); // Print column headers
268
269     // Loop through every patient and call their display() function
270     for (int i = 0; i < patientCount; i++)
271         patients[i].display();
272
273     cout << string(75, '-') << "\n";
274 }
275
276
277 // =====
278 // MENU 3A: bubbleSortByAge()
279 //
280 // HOW BUBBLE SORT WORKS:
281 // - Compare two ADJACENT (side-by-side) elements
282 // - If the left one is BIGGER, SWAP them
283 // - Do this across the whole array – the biggest value
284 //   "bubbles up" to the end like a bubble in water
285 // - Repeat for remaining unsorted portion
286 //
287 // EXAMPLE with ages: [45, 28, 34, 31]
288 // Pass 1: compare 45,28 ? swap ? [28,45,34,31]
289 //         compare 45,34 ? swap ? [28,34,45,31]
290 //         compare 45,31 ? swap ? [28,34,31,45] ? 45 bubbled to end
291 // Pass 2: compare 28,34 ? no swap
292 //         compare 34,31 ? swap ? [28,31,34,45]
293 // Pass 3: compare 28,31 ? no swap (swapped=false ? STOP early!)
294 //
295 // Time Complexity:
296 //   Best Case O(n)   – already sorted, detect with swapped flag
297 //   Average   O(n²)  – random data
298 //   Worst Case O(n²) – reverse sorted data
299 // =====
300 void bubbleSortByAge() {
301     // Outer loop: number of passes needed
302     for (int i = 0; i < patientCount - 1; i++) {
303         bool swapped = false; // Track if ANY swap happened this pass
304
305         // Inner loop: compare adjacent pairs
306         // After pass i, the last i elements are already sorted
307         // so we only go up to patientCount-1-i
308         for (int j = 0; j < patientCount - 1 - i; j++) {
309             if (patients[j].age > patients[j + 1].age) {
310                 // SWAP the two patients
311                 Patient temp      = patients[j];
312                 patients[j]       = patients[j + 1];
313                 patients[j + 1]   = temp;
314                 swapped = true;
315             }
316         }
317
318         // OPTIMIZATION: If no swap occurred in this entire pass,
319         // the array is already sorted – stop early
320         // This is what gives us O(n) best case
321         if (!swapped) break;
322     }
323
324     sortedByID = false; // Sorting by age breaks our ID ordering
325     cout << "Patients sorted by age successfully.\n";
326 }
327
328
329 // =====
330 // MENU 3B: selectionSortByPriority()

```

```

331 //
332 // HOW SELECTION SORT WORKS:
333 // - Divide array into SORTED (left) and UNSORTED (right) parts
334 // - Each pass: FIND the MINIMUM in the unsorted part
335 // - SWAP that minimum into the first position of unsorted part
336 // - The sorted part grows by 1 each pass
337 //
338 // EXAMPLE with priorities: [3, 1, 2, 1, 2]
339 // Pass 0: min in [3,1,2,1,2] is 1 at index 1 ? swap 3 and 1
340 //      ? [1, 3, 2, 1, 2]
341 // Pass 1: min in [3,2,1,2] is 1 at index 3 ? swap 3 and 1
342 //      ? [1, 1, 2, 3, 2]
343 // Pass 2: min in [2,3,2] is 2 at index 2 ? no swap needed
344 //      ? [1, 1, 2, 3, 2]
345 // ... and so on
346 //
347 // Time Complexity:  $O(n^2)$  in ALL cases (best, average, worst)
348 // Because the inner loop always runs fully to find the minimum
349 // =====
350 void selectionSortByPriority() {
351     for (int i = 0; i < patientCount - 1; i++) {
352         int minIdx = i; // Assume current position has minimum
353
354         // Scan the rest of the unsorted portion for a smaller priority
355         for (int j = i + 1; j < patientCount; j++) {
356             if (patients[j].priority < patients[minIdx].priority)
357                 minIdx = j; // Found new minimum
358         }
359
360         // Only swap if minimum is not already in correct position
361         if (minIdx != i) {
362             Patient temp = patients[i];
363             patients[i] = patients[minIdx];
364             patients[minIdx] = temp;
365         }
366     }
367
368     sortedByID = false;
369     cout << "Patients sorted by priority successfully.\n";
370 }
371
372 void sortMenu() {
373     cout << "\n===== SORT PATIENTS =====\n";
374     cout << "Sort by:\n";
375     cout << "1. Age (Bubble Sort)\n";
376     cout << "2. Priority (Selection Sort)\n";
377     cout << "Enter choice: ";
378     int choice;
379     cin >> choice;
380
381     if (choice == 1) bubbleSortByAge();
382     else if (choice == 2) selectionSortByPriority();
383     else cout << "Invalid choice.\n";
384 }
385
386 // =====
387 // MENU 4A: linearSearchByName()
388 //
389 // HOW LINEAR SEARCH WORKS:
390 // - Start at position 0 in the array
391 // - Check: does this patient's name match what we want?
392 // - If YES ? display it
393 // - If NO ? move to next position
394 // - Continue until end of array
395 //
396

```

```

397 // EXAMPLE: searching for "Marie" in [Alice, Eric, Marie, Jean]
398 // Check index 0: Alice == Marie? NO ? move on
399 // Check index 1: Eric == Marie? NO ? move on
400 // Check index 2: Marie == Marie? YES ? display!
401 // Check index 3: Jean == Marie? NO ? end
402 //
403 // Time Complexity:
404 // Best Case O(1) – target is the very first element
405 // Average O(n) – target is somewhere in the middle
406 // Worst Case O(n) – target is last or not found (checked all n)
407 // =====
408 void linearSearchByName() {
409     string target;
410     cout << "Enter patient name to search: ";
411     cin.ignore();
412     getline(cin, target);
413
414     bool found = false;
415     printHeader();
416
417     // Check every element one by one
418     for (int i = 0; i < patientCount; i++) {
419         if (patients[i].name == target) {
420             patients[i].display();
421             found = true;
422             // Note: we don't break – we show ALL patients with this name
423         }
424     }
425
426     if (!found)
427         cout << "No patient found with name: " << target << "\n";
428 }
429
430
431 // =====
432 // PRIVATE HELPER: sortByID()
433 // Sorts patients by their ID string before binary search
434 // "P001" < "P002" < "P010" etc. – string comparison works correctly
435 // =====
436 void sortByID() {
437     cout << "Sorting by ID first (required for Binary Search)... \n";
438     for (int i = 0; i < patientCount - 1; i++) {
439         for (int j = 0; j < patientCount - 1 - i; j++) {
440             if (patients[j].id > patients[j + 1].id) {
441                 Patient temp = patients[j];
442                 patients[j] = patients[j + 1];
443                 patients[j + 1] = temp;
444             }
445         }
446     }
447     sortedByID = true;
448     cout << "Array sorted by ID. \n";
449 }
450
451
452 // =====
453 // MENU 4B: binarySearchByID()
454 //
455 // HOW BINARY SEARCH WORKS:
456 // - Array MUST be sorted first (by ID in our case)
457 // - Look at the MIDDLE element
458 // - If it's our target ? FOUND!
459 // - If target > middle ? search only the RIGHT half (discard left)
460 // - If target < middle ? search only the LEFT half (discard right)
461 // - Repeat on the remaining half until found or empty
462 //

```

```

463 // EXAMPLE: find P004 in [P001, P002, P003, P004, P005, P006, P007]
464 //           lo=0, hi=6, mid=3 ? patients[3].id = P004 ? FOUND!
465 //
466 // EXAMPLE: find P006 in [P001, P002, P003, P004, P005, P006, P007]
467 //           lo=0, hi=6, mid=3 ? P004 < P006 ? lo = mid+1 = 4
468 //           lo=4, hi=6, mid=5 ? P006 == P006 ? FOUND!
469 //
470 // Time Complexity:
471 //   Best Case O(1)      - target is at the middle on first check
472 //   Average   O(log n) - e.g. 1000 patients needs at most 10 checks
473 //   Worst Case O(log n) - target not found
474 //
475 // WHY O(log n)?
476 //   Each step HALVES the search space:
477 //   n ? n/2 ? n/4 ? n/8 ? ... ? 1
478 //   That's log?(n) steps
479 // =====
480 void binarySearchByID() {
481     // REQUIREMENT: array must be sorted by ID
482     if (!sortedByID) {
483         cout << "\nBinary search requires sorted data.\n";
484         sortByID();
485     }
486
487     string target;
488     cout << "Enter Patient ID: ";
489     cin >> target;
490
491     int lo = 0; // Start of search range
492     int hi = patientCount - 1; // End of search range
493     bool found = false;
494
495     while (lo <= hi) {
496         // Find the middle index
497         // We write lo + (hi-lo)/2 instead of (lo+hi)/2
498         // to avoid integer overflow when lo+hi is very large
499         int mid = lo + (hi - lo) / 2;
500
501         if (patients[mid].id == target) {
502             // FOUND - display and stop
503             cout << "\nPatient found:\n";
504             printHeader();
505             patients[mid].display();
506             found = true;
507             break;
508         } else if (patients[mid].id < target) {
509             // Target is in the RIGHT half
510             lo = mid + 1;
511         } else {
512             // Target is in the LEFT half
513             hi = mid - 1;
514         }
515     }
516
517     if (!found)
518         cout << "Patient with ID '" << target << "' not found.\n";
519 }
520
521 void searchMenu() {
522     cout << "\n===== SEARCH PATIENT =====\n";
523     cout << "Search by:\n";
524     cout << "1. Name (Linear Search)\n";
525     cout << "2. ID   (Binary Search)\n";
526     cout << "Enter choice: ";
527     int choice;
528     cin >> choice;

```



```

529
530     if (choice == 1) linearSearchByName();
531     else if (choice == 2) binarySearchByID();
532     else cout << "Invalid choice.\n";
533 }
534
535
536 // =====
537 // MENU 5: updatePatient()
538 // Find a patient by ID and update one of their fields
539 // =====
540 void updatePatient() {
541     string searchID;
542     cout << "\n===== UPDATE PATIENT RECORD =====\n";
543     cout << "Enter Patient ID to update: ";
544     cin >> searchID;
545
546     // Linear search to find the patient
547     int foundIdx = -1; // -1 means "not found yet"
548     for (int i = 0; i < patientCount; i++) {
549         if (patients[i].id == searchID) {
550             foundIdx = i;
551             break; // Stop as soon as we find it
552         }
553     }
554
555     if (foundIdx == -1) {
556         cout << "Patient with ID '" << searchID << "' not found.\n";
557         return;
558     }
559
560     // Show current record before changing it
561     cout << "\nCurrent record:\n";
562     printHeader();
563     patients[foundIdx].display();
564
565     cout << "\nWhat would you like to update?\n";
566     cout << "1. Name\n";
567     cout << "2. Age\n";
568     cout << "3. Diagnosis\n";
569     cout << "4. Hospital\n";
570     cout << "5. Priority\n";
571     cout << "Enter choice: ";
572     int choice;
573     cin >> choice;
574
575     switch (choice) {
576     case 1:
577         cout << "Enter new name: ";
578         cin.ignore();
579         getline(cin, patients[foundIdx].name);
580         cout << "Name updated successfully.\n";
581         break;
582
583     case 2:
584         cout << "Enter new age: ";
585         cin >> patients[foundIdx].age;
586         while (patients[foundIdx].age < 0 || patients[foundIdx].age > 120) {
587             cout << "Invalid! Enter 0-120: ";
588             cin >> patients[foundIdx].age;
589         }
590         cout << "Age updated successfully.\n";
591         break;
592
593     case 3:
594         cout << "Enter new diagnosis: ";

```

```

595         cin.ignore();
596         getline(cin, patients[foundIdx].diagnosis);
597         cout << "Diagnosis updated successfully.\n";
598         break;
599
600     case 4: {
601         cout << "Available Hospitals:\n";
602         for (int i = 0; i < 5; i++)
603             cout << " " << hospitals[i].index << ". " << hospitals[i].name << "\n";
604         cout << "Enter new hospital index (1-5): ";
605         int hi;
606         cin >> hi;
607         while (hi < 1 || hi > 5) {
608             cout << "Invalid! Enter 1-5: ";
609             cin >> hi;
610         }
611         patients[foundIdx].hospital = getHospitalName(hi);
612         cout << "Hospital updated successfully.\n";
613         break;
614     }
615
616     case 5:
617         cout << "Enter new priority (1=Emergency, 2=Urgent, 3=Normal): ";
618         cin >> patients[foundIdx].priority;
619         while (patients[foundIdx].priority < 1 || patients[foundIdx].priority > 3) {
620             cout << "Invalid! Enter 1, 2, or 3: ";
621             cin >> patients[foundIdx].priority;
622         }
623         cout << "Priority updated successfully.\n";
624         break;
625
626     default:
627         cout << "Invalid choice. No changes made.\n";
628         return;
629 }
630 cout << "Patient record updated.\n";
631 }
632
633
634 // =====
635 // MENU 6: deletePatient()
636 // Find patient by ID, remove them from the array
637 //
638 // HOW DELETION FROM ARRAY WORKS:
639 // Array: [P001, P002, P003, P004, P005]
640 // Delete P003 (index 2):
641 // Shift left starting from index 2:
642 //   patients[2] = patients[3] ? [P001, P002, P004, P004, P005]
643 //   patients[3] = patients[4] ? [P001, P002, P004, P005, P005]
644 // patientCount-- ? [P001, P002, P004, P005] (count=4)
645 // The last duplicate is "ignored" since count says only 4 exist
646 // =====
647 void deletePatient() {
648     string searchID;
649     cout << "\n===== DISCHARGE PATIENT =====\n";
650     cout << "Enter Patient ID to discharge: ";
651     cin >> searchID;
652
653     // Find the patient
654     int foundIdx = -1;
655     for (int i = 0; i < patientCount; i++) {
656         if (patients[i].id == searchID) {
657             foundIdx = i;
658             break;
659         }
660     }

```

```

661
662     if (foundIdx == -1) {
663         cout << "Patient not found.\n";
664         return;
665     }
666
667     // Emergency patients need extra confirmation
668     if (patients[foundIdx].priority == 1) {
669         char confirm;
670         cout << "\n!!! WARNING !!!\n";
671         cout << patients[foundIdx].name << " is an EMERGENCY patient.\n";
672         cout << "Discharging an emergency patient could be dangerous.\n";
673         cout << "Are you absolutely sure? (y/n): ";
674         cin >> confirm;
675         if (confirm != 'y' && confirm != 'Y') {
676             cout << "Discharge cancelled. Patient kept in system.\n";
677             return;
678         }
679     }
680
681     // Save name before deletion for confirmation message
682     string name = patients[foundIdx].name;
683
684     // Shift all elements after foundIdx one position to the LEFT
685     for (int i = foundIdx; i < patientCount - 1; i++) {
686         patients[i] = patients[i + 1];
687     }
688     patientCount--; // One fewer patient
689
690     cout << name << " has been successfully discharged.\n";
691 }
692
693
694 // =====
695 // MENU 7: displayByHospital()
696 // Show only patients assigned to a chosen hospital
697 // =====
698 void displayByHospital() {
699     cout << "\n===== PATIENTS BY HOSPITAL =====\n";
700     cout << "Select hospital:\n";
701     for (int i = 0; i < 5; i++)
702         cout << " " << hospitals[i].index << ". " << hospitals[i].name << "\n";
703     cout << "Enter hospital index (1-5): ";
704     int idx;
705     cin >> idx;
706     while (idx < 1 || idx > 5) {
707         cout << "Invalid! Enter 1-5: ";
708         cin >> idx;
709     }
710
711     string hospName = getHospitalName(idx);
712     cout << "\nPatients assigned to " << hospName << ":\n";
713     printHeader();
714
715     bool found = false;
716     for (int i = 0; i < patientCount; i++) {
717         if (patients[i].hospital == hospName) {
718             patients[i].display();
719             found = true;
720         }
721     }
722
723     if (!found)
724         cout << "No patients currently assigned to " << hospName << ".\n";
725 }
726

```

```

727
728 // =====
729 // MENU 8: saveToFile()
730 // Writes all data to patients.txt and hospitals.txt
731 //
732 // FILE FORMAT (pipe-delimited for easy reloading):
733 // P001|Alice Uwase|28|CHUK|Malaria|1
734 // P002|Eric Nkurunziza|45|Butaro|Diabetes|3
735 //
736 // WHY PIPE '|' ?
737 // Patient names can have spaces ("Jean Baptiste")
738 // If we used spaces as delimiters, we'd confuse name spaces with field separators
739 // The pipe character | never appears in names or diagnoses
740 // =====
741 void saveToFile() {
742     // --- Save patients ---
743     ofstream pFile("patients.txt");
744     if (!pFile.is_open()) {
745         cout << "Error: Could not open patients.txt for writing.\n";
746         return;
747     }
748
749     // Write each patient as pipe-delimited row
750     for (int i = 0; i < patientCount; i++) {
751         pFile << patients[i].id << "|"
752             << patients[i].name << "|"
753             << patients[i].age << "|"
754             << patients[i].hospital << "|"
755             << patients[i].diagnosis << "|"
756             << patients[i].priority << "\n";
757     }
758     pFile.close();
759
760     // --- Save hospitals ---
761     ofstream hFile("hospitals.txt");
762     if (!hFile.is_open()) {
763         cout << "Error: Could not open hospitals.txt for writing.\n";
764         return;
765     }
766
767     hFile << "Index|Hospital Name\n";
768     for (int i = 0; i < 5; i++) {
769         hFile << hospitals[i].index << "|" << hospitals[i].name << "\n";
770     }
771     hFile.close();
772
773     cout << "Data saved successfully to patients.txt and hospitals.txt\n";
774 }
775
776
777 // =====
778 // loadFromFile()
779 // Called ONCE at startup – reads patients.txt back into array
780 //
781 // HOW stringstream WORKS:
782 // line = "P001|Alice Uwase|28|CHUK|Malaria|1"
783 // We create a stringstream from the line, then use getline(ss, field, '|')
784 // to read each piece separated by '|' into separate variables
785 // =====
786 void loadFromFile() {
787     ifstream file("patients.txt");
788     if (!file.is_open()) {
789         cout << "(No saved file found – starting fresh)\n";
790         return;
791     }
792

```

```

793     string line;
794     while (getline(file, line)) {
795         if (line.empty()) continue; // Skip blank lines
796
797         stringstream ss(line); // Turn the line into a stream
798         string id, name, ageStr, hospital, diagnosis, priorityStr;
799
800         // Extract each field separated by '|'
801         getline(ss, id, '|');
802         getline(ss, name, '|');
803         getline(ss, ageStr, '|');
804         getline(ss, hospital, '|');
805         getline(ss, diagnosis, '|');
806         getline(ss, priorityStr, '|');
807
808         // Convert string numbers to int
809         int age = stoi(ageStr);
810         int priority = stoi(priorityStr);
811
812         // Store the reconstructed patient
813         patients[patientCount++] = Patient(id, name, age, diagnosis, hospital, priority);
814
815         // Update idCounter to the highest number seen
816         // id.substr(1) removes the "P" ? "001" ? stoi gives 1
817         int num = stoi(id.substr(1));
818         if (num > idCounter) idCounter = num;
819     }
820
821     file.close();
822     cout << patientCount << " patients loaded from file.\n";
823 }
824
825
826 // =====
827 // run() - The main menu loop
828 // Keeps showing the menu until user picks 9 (Exit)
829 // =====
830 void run() {
831     loadFromFile(); // Load saved data on startup
832
833     int choice;
834     do {
835         cout << "\n\n===== \n";
836         cout << "    RWANDA HOSPITAL PATIENT MANAGEMENT SYSTEM  \n";
837         cout << "===== \n";
838         cout << " 1. Add new patient\n";
839         cout << " 2. Display all patients\n";
840         cout << " 3. Sort patients (by age or priority)\n";
841         cout << " 4. Search for a patient\n";
842         cout << " 5. Update patient record\n";
843         cout << " 6. Delete patient (discharge)\n";
844         cout << " 7. Display patients by hospital\n";
845         cout << " 8. Save data to file\n";
846         cout << " 9. Exit\n";
847         cout << "===== \n";
848         cout << "Enter choice: ";
849         cin >> choice;
850
851         switch (choice) {
852             case 1: addPatient(); break;
853             case 2: displayAll(); break;
854             case 3: sortMenu(); break;
855             case 4: searchMenu(); break;
856             case 5: updatePatient(); break;
857             case 6: deletePatient(); break;
858             case 7: displayByHospital(); break;

```

```

859         case 8: saveToFile();          break;
860         case 9: cout << "\nExiting system. Data saved. Goodbye!\n";
861                 saveToFile(); break;    // Auto-save on exit
862         default: cout << "Invalid choice! Please enter 1-9.\n";
863     }
864
865     } while (choice != 9);
866 }
867
868 }; // end class PatientSystem
869
870
871 // =====
872 // main() – Entry point of the program
873 // Creates ONE PatientSystem object and calls run()
874 // =====
875 int main() {
876     PatientSystem system; // Creates object – constructor runs here
877     system.run();         // Starts the menu loop
878     return 0;             // Program ends cleanly
879 }

```